Syntax Analyzer

Wednesday, March 25, 2026 9:00 PM

Definite Finite Automaton (DFA) by Keyword

Format:
`TOKEN: keyword`

```
KEYWORD_VAR: var
KEYWORD_IF: if
KEYWORD_ELSE: else
KEYWORD_WHILE: while
KEYWORD_FUNCTION: function
KEYWORD_RETURN: return
```

Definite Finite Automaton (DFA) by Operator

```
OP_SUM: +
OP_SUBST: -
OP_MULT: *
OP_DIV: /
OP_EQUALS: =
OP_IS_EQUAL: ==
OP_IS_DIFFERENT: !=
OP_LESS_THAN: <
OP_GREATER_THAN: >
```

Note: the images of the DFAs for each reserved word have been omitted from this document, as the definitions provided in this text are sufficient for documenting the parser.

Context-Free Grammars (CFG) by Expression Type

Considerations:

$letter = \{w : w \in \Sigma\} \mid \Sigma = ASCII\ alphabet$

$digit\ = \{n : n \in N\}$

General token for any type of expression:

Token: `ANY_EXPRESSION`

$A \rightarrow E \mid (A)$
$E \rightarrow$ `VAL_NUM` | `IDENTIFIER` | `OP_ALGEB` | `OP_LOGIC` | `INV_FUNCTION`

Notes: This token encompasses all types of expressions defined below. Note that this grammar also supports all expressions nested within parentheses. Example:

`(((OP_ALGEB)))` ∈ `ANY_EXPRESSION`

Tokens by expression type:

- ***Numeric values:***
  Token: `VAL_NUM`
  Integer or decimal

$N \rightarrow I\ P$
$I \rightarrow \text{digit}\ M$
$M \rightarrow \text{digit}\ M \mid \varepsilon$
$P \rightarrow .D \mid \varepsilon$
$D \rightarrow \text{digit}\ M$

Note: In our programming language, every numeric expression must have at least one digit after the decimal point if it is a floating-point value, and if it is an integer, it must not have a decimal point or any digits after the decimal point, and it must have at least one digit to be considered a numeric value.

- ***Identifiers:***
  Token: `IDENTIFIER`
  A combination of letters, digits and underscores; it must begin with a letter

  $I \rightarrow \text{letter}\ R$
  $R \rightarrow \text{letter}\ R \mid \text{digit}\ R \mid \_R \mid \varepsilon$

  Note: An identifier must begin with an ASCII letter and may optionally contain additional characters, which can only be letters, digits, or underscores (_). Note that in our grammar, we allow variables that end with underscores (e.g., `new_variable_`).

- ***Algebraic Operations:***
  Token: `OP_ALGEB`
  Addition `+`, subtraction `-`, multiplication `*`, division `/`

  $A \rightarrow N\ O\ N$
  $N \rightarrow$ `ANY_EXPRESSION`
  $O \rightarrow$ `OP_SUM` | `OP_SUBST` | `OP_MULT` | `OP_DIV`

  Recalling that `OP_ALGEB` $\subset$ `ANY_EXPRESSION`

- **Logical Operations:**
  Token: `OP_LOGIC`
  Logical operators (`<`,`>`,`==`,`!=`)

  $L \rightarrow A\ O\ A$
  $A \rightarrow$ `ANY_EXPRESSION`
  $O \rightarrow$ `OP_IS_EQUAL` | `OP_IS_DIFFERENT` | `OP_LESS_THAN` | `OP_GREATER_THAN`

  Recalling that `OP_LOGIC` $\subset$ `ANY_EXPRESSION`

- **Function Calls:**
  Token: `INV_FUNCTION`
  identifier(parameters)

  $F \rightarrow$ `IDENTIFIER` (P)
  $P \rightarrow$ `ANY_EXPRESSION` $N \mid \varepsilon$
  $N \rightarrow$ `,ANY_EXPRESSION` $N \mid \varepsilon$

  Recalling that `INV_FUNCTION` $\subset$ `ANY_EXPRESSION`

  Note: To call a function (but not necessarily declare it), you must first write the function name followed by its matching parentheses; you may optionally include parameters to be passed to the function in any format.

Context-Free Grammars (CFG) by Statement Type

General token for any type of statement:
Token: ANY_STATEMENT

$A \rightarrow$ DEF_VAR_STATEMENT | ASSIGN_STATEMENT | CONDITION_STATEMENT | LOOP_STATEMENT | DEF_FUNCTION

- **Definition of variables:**
  Token: DEF_VAR_STATEMENT
  Acceptance sample:
  ```
  var x = 5;
  var my_variable123 = myFunction(3 + 1, 50.5);
  var a;
  ```

  $S \rightarrow$ KEYWORD_VAR N
  $N \rightarrow$ IDENTIFIER O
  $O \rightarrow$ OP_EQUALS A | ;
  $A \rightarrow$ ANY_EXPRESSION ;

  Notes: Every variable declaration must begin with the keyword `var`, followed by the identifier for the variable and, optionally, the equals operator and the expression that will define the variable. Note that, in this way, the variable may or may not be assigned a value at the time of its definition.

- **Variable Assignment:**
  Token: ASSIGN_STATEMENT
  Acceptance sample:
  ```
  x = x + 1;
  ```

  $S \rightarrow$ IDENTIFIER O
  $O \rightarrow$ OP_EQUALS A
  $A \rightarrow$ ANY_EXPRESSION ;

  Notes: Every variable assignment must begin with the variable identifier, followed by the equal sign and the expression whose value will be assigned to the variable. In this case, the equal sign and the expression are required.

- **If-then[-else] Statements:**
  Token: CONDITION_STATEMENT
  Acceptance sample:
  ```
  if (x > 0) { ... }
  if (x > 0) { ... } else { ... }
  ```

  $S \rightarrow$ KEYWORD_IF (C) $\{B\}$ O
  $C \rightarrow$ OP_LOGIC
  $B \rightarrow$ ANY_STATEMENT M | ε
  $O \rightarrow$ KEYWORD_ELSE $\{B\}$ | ε

  Recalling that CONDITION_STATEMENT $\subset$ ANY_STATEMENT

  Notes: Every conditional block must begin with the reserved word `if` along with the condition to be tested, enclosed in a pair of balanced parentheses, followed by a pair of balanced curly braces, the body of which must contain one or more numbers and types of statutes (including the possibility of nesting another conditional block). Note that for each conditional definition `if`, It is optional to place only one block `else`, In this sense, to place nested conditional blocks, this structure must be followed:
  ```
  if (condition) { ... } else { if (condition) { ... } }
  ```

- **Loop "while" statements:**
  Token: LOOP_STATEMENT
  Acceptance sample:
  ```
  while (x < 10) { ... }
  ```

  $S \rightarrow$ KEYWORD_WHILE $(C)\,\{B\}$
  $C \rightarrow$ OP_LOGIC
  $B \rightarrow$ ANY_STATEMENT $M \mid \varepsilon$

  Recalling that LOOP_STATEMENT $\subset$ ANY_STATEMENT

  Notes: In accordance with the philosophy of conditional status, all cyclical statements `while` must begin with the keyword `while` along with the logical operation it will use to perform the iteration; this logical operation must be enclosed in a balanced pair of parentheses. Likewise, the loop `while` must contain a body enclosed by a pair of balanced curly braces, which will contain one or more statutes (including the possibility of nesting cycles `while`).

- **Function Definition Statement:**
  Token: DEF_FUNCTION
  Acceptance sample:
  ```
  function add(a, b) { return a + b; }
  ```

  $S \rightarrow$ KEYWORD_FUNCTION N
  $N \rightarrow$ IDENTIFIER $(P)\,\{B\}$
  $P \rightarrow$ IDENTIFIER $M \mid \varepsilon$
  $M \rightarrow$, IDENTIFIER $M \mid \varepsilon$
  $B \rightarrow$ ANY_STATEMENT $B \mid$ KEYWORD_RETURN $R \mid \varepsilon$
  $R \rightarrow$ ANY_EXPRESSION ;

  Notes: Every function definition must begin with its signature, which has the following structure:
  Reserved word `function`, a function identifier and a single pair of balanced parentheses containing zero or more identifiers (local constants of the function).
  Once the function signature has been declared, you must write only one pair of balanced curly braces containing one or more statements (whether variable declarations, variable assignments, conditional blocks, or loop blocks) and, optionally, a return statement that must begin with the keyword `return`, followed by any type of expression (identifier, numeric value, algebraic operation, logical operation, or function call) and a semicolon '`;`' at the end of the statement.
  Note that, unlike the function call expression (INV_FUNCTION), The function signature does not allow any expressions other than identifiers, and the function parameters are defined using only variable names.

General Grammar

Any program can be defined as a list of rules

$P \rightarrow$ ANY_STATEMENT $P \mid$ ANY_STATEMENT